# Specializing ML-DSA-44 Verification for Secure Boot on Cortex-M4

## A Didactic Walk-through of Ten Progressive Optimizations

2026-05-11


**Abstract**

We present a custom verifier for ML-DSA-44 (NIST FIPS 204) targeted at secure-boot deployments on the ARM Cortex-M4. Starting from `pqm4`'s `m4f` backend — the fastest publicly available Cortex-M4 verifier we benchmarked, beating both `mldsa-native`'s portable C90 and the round-3 Dilithium2 implementation in `FasterKyberDilithiumM4` on this metric — we apply ten layered optimizations and reduce per-verify cost from 1,261,555 to 241,756 instructions retired on the Cortex-M4 ISA — a **5.22**× speedup. The optimizations exploit two observations that hold for secure boot but not for general ML-DSA verification: (i) the public key is fixed at provisioning time, so any quantity that depends only on `pk` may be precomputed and stored in flash; (ii) all verifier inputs are public, so the constant-time discipline of `pqm4` is no longer load-bearing.

The verifier is *not* a drop-in replacement for a general ML-DSA-44 verify routine: it is hard-specialized to the provisioned public key and rejects any input with non-empty context. On the secure-boot input subset (ML-DSA-44, the provisioned `pk`, $|\text{ctx}| = 0$), nine of the ten variants are accept/reject-equivalent to FIPS 204 Algorithm 3 *by construction* (every transformation either precomputes a deterministic function of `pk` or refactors C code without changing its behaviour). One variant (`v9`, which removes a polynomial reduction we conjecture to be redundant under our pre-invNTT bounds) has an open soundness question we have not closed by worst-case analysis; we verify it empirically on the test vector `mldsa-native` produces. We give both high-level intuition and the precise transformations applied at each step, with measurements at every checkpoint reproducible from a small QEMU harness.

**Headline:** ML-DSA-44 signature verification on Cortex-M4 becomes ∼34× cheaper than pure-C P-256 ECDSA verify and ∼51× cheaper than the hand-asm `p256-m`.


# 1 Introduction

The migration from elliptic-curve cryptography (ECC) to post-quantum cryptography is now operational policy: NIST published the first three post-quantum standards in August 2024, including FIPS 204 (ML-DSA, a digital signature algorithm based on the Dilithium [1] module lattice construction). Secure-boot is among the first deployment domains where the new standard matters: the public verification key is provisioned into a device at manufacture, then the bootloader verifies a firmware signature on every boot until end-of-life. Verifier cost is paid every boot; key size is paid once.

ML-DSA's verify path is dominated by polynomial arithmetic over the ring $R_q = \mathbb{Z}_q[X]/(X^{256}+1)$, $q = 2^{23} - 2^{13} + 1 = 8{,}380{,}417$, with heavy use of the Number-Theoretic Transform (NTT). Compared to a P-256 ECDSA verify on the same Cortex-M4, an off-the-shelf ML-DSA-44 verify in C is already ∼3–4× cheaper; with the hand-tuned `pqm4` [3] M4 asm it becomes ∼6× cheaper. This paper asks how much further the cost can fall when one exploits the *fixed-public-key, all-public-data* setting that defines secure boot.

The contribution is twofold. First, we measure ten ML-DSA-44 verifier variants on a single bare-metal Cortex-M4 testbench, with reproducible instruction-retired counts. Second, and

primarily, we deliver a didactic exposition: each optimization is presented with its high-level justification, the precise FIPS 204 step it touches, and its measured impact, so a practitioner can decide which transformations apply to their threat model and which do not. We are explicit about what we do *not* skip — the FIPS-204-mandated checks on $z$'s norm and the hint encoding remain in place — because for secure boot the correctness-of-verify property is the entire load-bearing assumption.

# 2 Background

This section gives just enough lattice-cryptography to follow the verification anatomy in Section 3. Readers fluent in module-LWE and the Dilithium structure may skim.

## 2.1 Module-LWE and the Ring $R_q$

Let $q$ be a prime and $n$ a power of two. Define the *negacyclic* ring

$$R_q \;:=\; \mathbb{Z}_q[X]/(X^n+1).$$

Elements of $R_q$ are polynomials of degree $< n$ with coefficients in $\mathbb{Z}_q$; multiplication is polynomial multiplication modulo $X^n+1$, which under our parameters ($n = 256$, $q = 2^{23} - 2^{13} + 1$) admits a fast $n \log n$ Number-Theoretic Transform.

The *module* variant of the Learning-With-Errors problem (M-LWE) fixes ranks $k, \ell \in \mathbb{N}$ and asks: distinguish

$$(\mathbf{A},\, \mathbf{As}+\mathbf{e}) \quad \text{from} \quad (\mathbf{A},\, \mathbf{u})$$

where $\mathbf{A} \in R_q^{k\times\ell}$ is uniform, $\mathbf{s} \in R_q^{\ell}$ and $\mathbf{e} \in R_q^{k}$ are short (coefficients in $\{-\eta, \ldots, \eta\}$ for a small $\eta$), and $\mathbf{u}$ is uniform in $R_q^k$. Both Kyber (now ML-KEM) and Dilithium (now ML-DSA) reduce their security to M-LWE and its sibling M-SIS.

## 2.2 Kyber (ML-KEM) at a Glance

ML-KEM (FIPS 203) is a key-encapsulation mechanism whose IND-CPA core is a ring-flavoured Regev-style PKE: the public key contains an M-LWE sample $\mathbf{t} = \mathbf{As} + \mathbf{e}$; encapsulation draws short $(\mathbf{r}, \mathbf{e}_1, e_2)$ and computes a ciphertext of the form $(\mathbf{u}, v) = (\mathbf{A}^\top\mathbf{r} + \mathbf{e}_1,\ \mathbf{t}^\top\mathbf{r} + e_2 + \delta(m))$ where the message $m \in \{0,1\}^n$ is lifted by $\delta(m)_i := m_i \cdot ((q+1)/2)$ to put each bit either at 0 or at the integer closest to $q/2$ in $\mathbb{Z}_q$. Decapsulation recovers $m_i$ by checking whether $(v - \mathbf{s}^\top\mathbf{u})_i$ is closer to 0 or to $(q+1)/2$, which succeeds whenever the error term $\mathbf{e}^\top\mathbf{r} - \mathbf{s}^\top\mathbf{e}_1 + e_2 - \mathbf{s}^\top\mathbf{e}$ has every coefficient of magnitude $< q/4$. Kyber additionally *compresses* the ciphertext and public key by truncating each coefficient to a few high-order bits (we elide that step here for the sketch), then wraps the IND-CPA scheme in the Fujisaki–Okamoto transform to achieve IND-CCA. We do not optimize Kyber in this work, but the $\mathbf{As} + \mathbf{e}$ pattern and the same underlying ring recur in Dilithium below.

## 2.3 Dilithium (ML-DSA) Construction

ML-DSA is a *Fiat–Shamir-with-aborts* signature: a $\Sigma$-protocol proving knowledge of a short M-LWE secret, made non-interactive by deriving the challenge via a random oracle (SHAKE-256). The construction's three signature levels are parametrized as in Table 1.

For ML-DSA-44 a public key occupies $32 + k \cdot 320 = 1{,}312$ bytes (rho plus the high-order $t_1$ of $\mathbf{t} = \mathbf{As}_1 + \mathbf{s}_2$ truncated by Power2Round) and a signature occupies $32 + \ell \cdot 576 + (\omega + k) = 2{,}420$ bytes (the challenge hash $\tilde{c}$, the masked response $\mathbf{z}$, and the hint $\mathbf{h}$).

Table 1: ML-DSA parameter sets (FIPS 204, §4 Table 1).

| | $q$ | $n$ | $d$ | $\tau$ | $\lambda$ | $\gamma_1$ | $\gamma_2$ | $(k,\ell)$ | $\eta$ | $\omega$ |
|---|---|---|---|---|---|---|---|---|---|---|
| ML-DSA-44 | $2^{23}-2^{13}+1$ | 256 | 13 | 39 | 128 | $2^{17}$ | $(q-1)/88$ | $(4,4)$ | 2 | 80 |
| ML-DSA-65 | „ | 256 | 13 | 49 | 192 | $2^{19}$ | $(q-1)/32$ | $(6,5)$ | 4 | 55 |
| ML-DSA-87 | „ | 256 | 13 | 60 | 256 | $2^{19}$ | $(q-1)/32$ | $(8,7)$ | 2 | 75 |

### 2.4 The Number-Theoretic Transform

Because $q-1 = 2^{23}-2^{13}$ is divisible by $2n = 512$, $\mathbb{Z}_q$ contains a primitive $2n$-th root of unity $\zeta$ (with $\zeta = 1753$ being the canonical choice in Dilithium). The *negacyclic* NTT evaluates a polynomial $a(X) \in R_q$ at the $n$ odd powers of $\zeta$:

$$\widehat{a}_i \;=\; a(\zeta^{2i+1}) \pmod q, \quad i = 0, \ldots, n-1.$$

Then $a \cdot b$ in $R_q$ equals $\mathrm{NTT}^{-1}(\widehat{a} \odot \widehat{b})$ where $\odot$ is the coefficient-wise product. The forward and inverse transforms run in $\mathcal{O}(n \log n)$ ring operations via the Cooley–Tukey/Gentleman–Sande structure, so the overall cost of a ring multiplication drops from $\mathcal{O}(n^2)$ schoolbook to $\mathcal{O}(n \log n)$ butterflies plus $n$ Montgomery pointwise multiplies. The `pqm4 m4f` backend implements the entire transform in hand-tuned Thumb-2 assembly, scheduling loads and stores to overlap with the M4's multi-cycle 32×32 multiplies; we treat it as a black box throughout.

### 2.5 Montgomery Arithmetic

Multiplications modulo $q$ avoid an expensive division by working in *Montgomery form*: integers are represented by $\bar{a} := aR \bmod q$ for a fixed $R$ coprime to $q$. We use $R = 2^{32}$ throughout. The *Montgomery reduction* of $T \in [0, qR)$ returns

$$\mathrm{Redc}(T) \;:=\; (T + m\,q)/R \;\equiv\; TR^{-1} \pmod q, \qquad m \;:=\; T \cdot q' \bmod R,$$

where $q'$ is the unique element of $\mathbb{Z}_R$ satisfying $q\,q' \equiv -1 \pmod R$ (the choice of sign makes $T + m\,q$ divisible by $R$ so the quotient is exact). The output sits in $[0, 2q)$, which a single conditional subtraction (or no subtraction at all, if downstream code admits a slightly-larger input range) brings into $[0, q)$. For Dilithium $q = 8{,}380{,}417$ and $R = 2^{32}$, pqm4 precomputes $q' = \texttt{0xFC7FDFFF} = -q^{-1} \bmod 2^{32}$.

The *Montgomery multiplication* of $\bar{a}, \bar{b} \in [0, q)$ is $\mathrm{Mul}(\bar{a}, \bar{b}) := \mathrm{Redc}(\bar{a} \cdot \bar{b})$, which yields $\overline{ab} = (ab)R \bmod q$ — i.e. the product stays in Montgomery form. In Thumb-2 the M4 implementation is three multiplies and a 64-bit add: `SMULL` of $\bar{a} \cdot \bar{b}$ producing a 64-bit $(T_{\mathrm{lo}}, T_{\mathrm{hi}})$ in a register pair, `MUL` of $T_{\mathrm{lo}} \cdot q'$ giving $m$, then `SMLAL` which accumulates $m \cdot q$ into the same register pair so that $T_{\mathrm{hi}}$ already holds $\mathrm{Redc}(T)$ on output. On Cortex-M4 `SMULL` and `SMLAL` are each 3–5 cycles and `MUL` is 1 cycle, so a full Montgomery multiplication retires in ∼8–11 cycles depending on operand magnitudes. Throughout this paper, "in NTT domain" should be read as "in NTT-Montgomery domain", matching the convention pqm4 uses.

## 3 ML-DSA-44 Verification, Anatomically

We rephrase FIPS 204's ML-DSA.Verify (Algorithm 3) and ML-DSA.Verify_internal (Algorithm 8) jointly as Algorithm 1 so that one walk-through covers both the public-API formatting layer and the lattice core. The comments tag each step with the implementation phase whose cost we will later measure and attack.

**Algorithm 1** ML-DSA verification (combined view of FIPS 204 Alg. 3 and Alg. 8).

**Require:** Public key $\mathsf{pk} = (\rho, t_1)$, message $M$, context string ctx with $|\text{ctx}| \leq 255$, signature $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$.

**Ensure:** $\top$ if $\sigma$ is a valid signature on $M$ under $\mathsf{pk}$, else $\perp$.

$(\tilde{c}, \mathbf{z}, \mathbf{h}) \leftarrow \textsc{UnpackSig}(\sigma)$ ▷ 18-bit unpack of $\mathbf{z}$; sparse hint decode; fail-on-malformed
**if** $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$ **then return** $\perp$
**end if** ▷ spec-mandated norm validation
$\mathbf{A} \leftarrow \text{ExpandA}(\rho) \in R_q^{k \times \ell}$ ▷ $k\ell$ **rejection-sampled** polynomials, output in NTT domain
$tr \leftarrow \text{SHAKE}_{256}(\mathsf{pk}, 64)$ ▷ 64-byte hash of the bit-packed public key
$M' \leftarrow \texttt{0x00} \,\|\, \text{IntegerToBytes}(|\text{ctx}|, 1) \,\|\, \text{ctx} \,\|\, M$ ▷ pure ML-DSA prefix
$\mu \leftarrow \text{SHAKE}_{256}(tr \,\|\, M', 64)$
$c \leftarrow \textsc{SampleInBall}(\tilde{c})$ ▷ $\tau$-sparse $\{-1, 0, +1\}$ polynomial via a SHAKE-256-seeded shuffle
$\hat{c} \leftarrow \text{NTT}(c)$; $\hat{\mathbf{z}} \leftarrow \text{NTT}(\mathbf{z})$
$\hat{\mathbf{w}}' \leftarrow \mathbf{A} \cdot \hat{\mathbf{z}} - \hat{c} \cdot \text{NTT}(2^d \cdot t_1)$ ▷ matrix–vector and one scaling, in NTT domain
$\mathbf{w}'_{\text{approx}} \leftarrow \text{UseHint}(\mathbf{h}, \text{NTT}^{-1}(\hat{\mathbf{w}}'))$ ▷ reconstruct HighBits with hint correction
$\tilde{c}' \leftarrow \text{SHAKE}_{256}(\mu \,\|\, \text{w1Encode}(\mathbf{w}'_{\text{approx}}), 2\lambda/8)$
**return** $\tilde{c}' \stackrel{?}{=} \tilde{c}$

**Where the cost goes.** Rather than estimate per-phase shares from instruction analysis, we report the cost of each implementation phase as the *measured difference* between two of our verifier variants (Section 5). Each variant either keeps or removes exactly one phase, so the delta is the cost of that phase in the baseline. The largest contributor is ExpandA, which generates the matrix $\mathbf{A}$ by rejection-sampling sixteen polynomials from SHAKE-128 streams keyed by $\rho \,\|\, j \,\|\, i$:

Table 2: Cost of each verify phase that we are able to switch off in *some* sb-mldsa44-v$N$ variant, measured directly as the per-verify instruction-retired delta between consecutive variants. Total verify cost in pqm4 `m4f` (v1) is 1,261,555 instructions; cumulative removals account for 1,019,799, leaving 241,756 in v10. Phases not represented here (the four NTTs on $\mathbf{z}$, the inverse NTT, the matrix–vector pointwise-Mont multiplications, the $\mu$ and $\tilde{c}'$ Keccak permutations not tied to pk, and the spec-mandated unpacks and checks) remain on the critical path.

| Phase removed | Delta source | Insns retired |
|---|---|---|
| ExpandA($\rho$) (16 SHAKE-128 rejection-sampling streams) | v1−v2 | 851,263 |
| unpack($t_1$) + shiftL$_{2^d}$ + NTT on $t_1$ | v2−v3 | 42,273 |
| SHAKE$_{256}$(pk, 64) → $tr$ | v3−v4 | 99,599 |
| zero-init + sub-rate XOR of $tr \,\|\,$ 0x00 0x00 | v4−v5 | 476 |
| $K$ redundant walks of the hint encoding | v5−v6 | 2,484 |
| three-pass caddq → UseHint → w1Encode | v6−v7 | 8,901 |
| separate unpack/chknorm scans over $\mathbf{z}$ | v7−v8 | 8,771 |
| poly_reduce before NTT$^{-1}$ (redundant) | v8−v9 | 5,412 |
| inc-API plumbing around $\tilde{c}'$ Keccak | v9−v10 | 620 |
| **cumulative removal** | v1−v10 | **1,019,799** |

**Observation.** ExpandA alone is 851,263/1,261,555 = 67.5% of the entire pqm4 `m4f` verify — which is precisely where the biggest payoff hides for a fixed-pk deployment.

# 4 The Secure-Boot Threat Model

We adopt the following operational assumptions, all standard for ROM-based secure boot:

1. **Fixed public key.** A single pk is provisioned at manufacturing time and is the only key the verifier ever consults. Any quantity that is a deterministic function of pk may be computed once at provisioning, stored in flash, and read on every boot.

2. **All inputs are public.** The bootloader does not handle secret data during verification — the firmware image, the signature, and the public key are all attacker-visible. The constant-time discipline that protects ML-DSA *signing* from side-channel attacks is therefore unnecessary on the verify side.

3. **Conformant on the secure-boot input subset.** Define the secure-boot input subset $\mathcal{I}_{\text{sb}}$ as the set of $(\mathsf{pk}, M, \text{ctx}, \sigma)$ tuples where pk equals the provisioned key and $|\text{ctx}| = 0$. The verifier reproduces the accept/reject decision of FIPS 204 Algorithm 3 on every input in $\mathcal{I}_{\text{sb}}$; on inputs outside this subset (different pk, or non-empty ctx) it deliberately rejects. Skipping any of the algorithm's validation steps (norm bound on $\mathbf{z}$, well-formedness of $\mathbf{h}$, monotone hint encoding) is *not* permitted within $\mathcal{I}_{\text{sb}}$. Internal representations and arithmetic detours that are unobservable at the accept/reject boundary (e.g. working entirely in NTT-Montgomery domain, or precomputing $\text{NTT}(\mathbf{A})$) are permitted. The one case in this series where we have not closed the equivalence proof even on $\mathcal{I}_{\text{sb}}$ is v9's skipped pre-invNTT reduction; we flag it prominently rather than asserting full conformance there.

4. **Empty context.** The bootloader signs without a context string ($|\text{ctx}| = 0$). This is the only message-shape assumption the optimized verifier actually *bakes in*: the 2-byte `0x00 0x00` prefix becomes part of the pre-warmed Keccak template at v5, and the verifier explicitly rejects inputs with non-zero $|\text{ctx}|$. The message itself flows through the verifier with an arbitrary length `mlen` (in our benchmark, `mlen` = 32 because the bootloader signs over SHA-256 of the image, but no version specializes around the value 32).

We adopt the convention that "do not cheat" means: every value the verifier outputs on inputs in $\mathcal{I}_{\text{sb}}$ is the value FIPS 204 would compute on the same inputs. Pre-storing $\text{NTT}(\mathbf{A})$ in flash is legitimate; hardcoding the verify result is not.

**Where precomputes live.** In the bench harness all precomputed tables live in `.bss` (RAM-backed `static` arrays) so the QEMU run is self-contained. A real secure-boot deployment would place them in flash; on a device with flash wait-states, that changes the read-side cost of every verify by the wait-state-times-cache-miss-rate factor. Our instruction-retired metric does not model this, so the absolute cycle counts on real silicon will be slightly higher than a naive `insns` → `cycles` translation suggests when precomputes are in flash; the relative ranking of variants is unaffected.

# 5 Optimizations

We now walk through the ten progressive variants v1 . . . v10. Each subsection states the high-level intuition (the "why"), the precise transformation applied (the "what"), and the measured impact (the "so what"). Throughout, the baseline is v1: the unmodified pqm4 `m4f` verify with secure-boot–shaped inputs. The cumulative speedup over v1 is tracked in Table 3 (Section 6).

### 5.1 v1 — Baseline

A direct call to `pqm4`'s `crypto_sign_verify_ctx` with $M = h$ (32 bytes) and ctx $= \varepsilon$ measures **1,261,555** instructions retired. This is ∼0.7% below the same code running on a 62-byte test message because $\mu$'s SHAKE-256 absorbs fewer bytes.

### 5.2 v2 — Precompute $\hat{\mathbf{A}} \in R_q^{k\times\ell}$

**Why.** ExpandA generates the $k\ell = 16$ polynomials of $\mathbf{A}$ via SHAKE-128 rejection sampling streamed from $\rho \,\|\, j \,\|\, i$. Because $\rho$ is the first 32 bytes of pk, and pk is fixed at provisioning, the entire matrix $\mathbf{A}$ is a constant. Storing it in flash — as $k \cdot \ell \cdot n \cdot 4 = 16\,\text{KB}$ of 32-bit Montgomery-domain NTT coefficients — eliminates the dominant cost of verify.

**What.** We add a one-shot `sb_v2_precompute(pk)` that calls `poly_uniform` for every $(i, j)$ pair and stores the result in a static `poly sb_A_NTT[K][L]`. The verify hot loop reads `&sb_A_NTT[i][j]` where the baseline called `expand_mat_elem`.

**So what.** v2 costs **410,292** instructions per verify — a **3.07**× speedup and a **−67.5**% reduction. The precompute itself costs 859,570 instructions, paid once.

### 5.3 v3 — Precompute $\mathrm{NTT}(2^d \cdot t_1)$

**Why.** The verifier needs $\hat{c} \cdot \mathrm{NTT}(2^d t_1)$ on every verify, but only $\hat{c}$ depends on the signature. The factor $\mathrm{NTT}(2^d t_1)$ depends only on pk (since $t_1$ is the Power2Round-truncated component of pk) and can be moved to precompute.

**What.** For each row index $k$, v3's precompute does `unpack_pk_t1` (10-bit unpack of 256 coefficients from the packed section of pk), `poly_shiftl` (multiply by $2^{13}$), and `poly_ntt`, storing the result in `poly sb_t1_ntt[K]`. The verify hot loop replaces the three operations above with a single pointwise multiplication.

**So what.** v3 costs **368,019** instructions (**3.43**× over v1; −10.3% over v2). The 4 KB precompute is small.

### 5.4 v4 — Precompute $tr = \mathrm{SHAKE}_{256}(\mathbf{pk}, 64)$

**Why.** The message representative $\mu = \mathrm{SHAKE}_{256}(tr \,\|\, \texttt{0x00} \,\|\, \mathrm{IntegerToBytes}(|\mathrm{ctx}|, 1) \,\|\, \mathrm{ctx} \,\|\, M, 64)$ is computed in pqm4 by first hashing the whole 1312-byte pk into $tr$, then absorbing $tr$ and the rest into a second SHAKE-256. Hashing pk with the rate-136 SHAKE-256 absorbs nine full blocks ($9 \cdot 136 = 1224$ bytes) followed by an 88-byte partial block; the spec-mandated finalize then runs one more permutation. So **nine absorb permutations plus one finalize permutation** are spent per verify on a quantity that depends only on pk.

**What.** The precompute stores `static uint8_t sb_tr_pk[TRBYTES = 64]`. The verify path absorbs `sb_tr_pk` directly instead of calling `shake256(tr, 64, pk, 1312)`.

**So what.** **268,420** instructions (**4.70**×; −27.1% over v3). The 64 bytes of flash buys the 10 Keccak permutations and the surrounding SHAKE-256 plumbing.

### 5.5 v5 — Pre-Warmed Keccak State for $\mu$

**Why.** After `v4`, the $\mu$ computation initializes a SHAKE-256 state, XORs in $tr$ (64 bytes), then XORs in the 2-byte domain separator `(0x00, |ctx|)`. Because $|\text{ctx}| = 0$ is fixed for secure boot, the post-prefix state is also constant: a 208-byte struct (the 200-byte Keccak state plus an 8-byte position counter holding the value 66).

**What.** The precompute walks a SHAKE-256 incremental state up to having $tr \parallel$ `0x00` $\parallel$ `0x00` absorbed, and stores the resulting struct. Verify `memcpy`s the template, absorbs the 32-byte message, finalizes, squeezes.

**So what.** A near-wash: **267**,**944** instructions (**4.71**×; −476 instructions over `v4`). On Cortex-M4 the cost of `memcpy`ing 208 bytes happens to almost equal the cost of zero-initializing the state and XORing the 66 prefix bytes. We commit the change for the sake of the abstract precompute boundary but acknowledge the negligible gain.

### 5.6 v6 — Decode the Hint Vector Once

**Why.** pqm4's `unpack_sig_h(&h, idx, sig)` extracts the hint polynomial for one row, but to validate the strong-unforgeability monotone-index requirement cumulatively it walks all $k$ rows of the encoded hint block on every call. Calling it $k$ times therefore visits the encoding $k^2$ times.

**What.** A single function `sb_v6_unpack_sig_h_all(hs, sig)` fills every `hs[i]` in one sweep, zero-fills $kn$ coefficients, validates the encoding once, and stores hint bits at the indicated coefficient positions.

**So what.** **265**,**460** instructions (**4.75**×; −2,484). Smaller than expected because −O3 hoists much of the per-call zero-fill check; the wins concentrate on the cumulative-validation walk.

### 5.7 v7 — Fuse `caddq + UseHint + polyw1_pack`

**Why.** The per-row verify tail traverses the 256-coefficient $\mathbf{w}'$ three times in succession: `poly_caddq` to re-center coefficients into $[0, q)$, `poly_use_hint` to reconstruct the high bits with $\mathbf{h}$, and `polyw1_pack` to compress to 6-bit-per-coefficient packed output. Each pass loads and stores 1 KB of memory; total redundant I/O per row is ∼3 KB.

**What.** A single `sb_v7_caddq_useHint_pack(packed, w, h)` processes coefficients in groups of four (the granularity of the 4-coeffs-in-3-bytes packing layout for $\gamma_2 = (q-1)/88$). Per coefficient it inlines `caddq`, the integer-arithmetic HighBits decomposition

$$a_1 \;=\; \left(\lfloor (a+127)/2^7 \rfloor \cdot 11275 + 2^{23}\right) \gg 24, \qquad a_1 \leftarrow 0 \text{ if } a_1 \geq 44,$$

the hint application $a_1 \leftarrow (a_1 \pm 1) \bmod 44$ on $\mathbf{h}$-marked positions, and the bit-shifting that packs four six-bit $a_1$ values into three bytes. The magic constant $11{,}275 = \lfloor 2^{31}/(2\gamma_2) \rfloor$ with $2\gamma_2 = (q-1)/44 = 190{,}464$ is a strength-reduced reciprocal of $2\gamma_2$ scaled to $2^{31}$; combined with the $\gg 7$ preconditioning and the $+2^{23}$ rounding offset before the outer $\gg 24$, the formula computes round-to-nearest-multiple-of-$2\gamma_2$ (i.e. the centred-representative HighBits as Dilithium defines it, not a floor) on every $a \in [0, q)$. The final clamp $a_1 \leftarrow 0$ if $a_1 \geq 44$ folds the wrap-around case for $a$ very close to $q-1$.

**So what.** **256**,**559** instructions (**4.92**×; −8,901). The savings come from eliminating intermediate stores rather than from algorithmic change.

### 5.8 v8 — Fuse Unpack and Norm-Check on z

**Why.** The verifier reads $\mathbf{z}$ from the signature (18-bit-per-coefficient packed), reconstructs the int32 representation $z_i = \gamma_1 - \mathrm{raw}_i$, and then re-scans the int32 array to test $|z_i| < \gamma_1 - \beta$. Two passes over $\ell n = 1024$ coefficients.

**What.** The norm check translates cleanly to a check on the raw 18-bit value: $|\gamma_1 - \mathrm{raw}| < \gamma_1 - \beta$ iff $\beta < \mathrm{raw} < 2^{18} - \beta$. For ML-DSA-44 with $\beta = 78$, the predicate $\mathrm{raw} \leq 78 \ \vee \ \mathrm{raw} \geq 262{,}066$ identifies an out-of-range coefficient. The fused `sb_v8_unpack_sig_z_chknorm` writes the int32 only after the predicate passes for all four coefficients of a group.

**So what.** **247,788** instructions (**5.09**×; −8,771). One full pass over 1 KB of int32 data eliminated.

**Aside.** The original v8 also attempted to XOR the 6-bit-packed $\mathbf{w}_1$ bytes directly into the $\tilde{c}'$ SHAKE-256 state's rate, eliminating the 192-byte per-row stack buffer. The implementation produced a different $\tilde{c}'$ than the inc-API reference; the bug appears to be a write-barrier issue between byte writes through a `(uint8_t *)` alias and the subsequent `KeccakF1600_StatePermute((uint64_t *))` call, but I was unable to isolate it inside the time budget. The win is partially recovered in `v10`.

### 5.9 v9 — Skip `poly_reduce` Before $\mathrm{NTT}^{-1}$

**Why.** After pqm4's `poly_pointwise_montgomery` / `poly_pointwise_acc_montgomery`, each NTT-Montgomery coefficient sits in roughly $(-q, q]$. After accumulating $\ell = 4$ such products in the matrix–vector inner loop and subtracting one more product for $\hat{c} \cdot \mathrm{NTT}(2^d t_1)$, an a-priori bound is $|\hat{w}'_i| \leq 5q$, i.e. comfortably inside 32-bit signed arithmetic. pqm4 calls `poly_reduce` next to compress the range toward $[-(q-1)/2, (q-1)/2]$ before feeding $\mathrm{NTT}^{-1}$; empirically the `m4f` invNTT-tomont yields the correct value on the un-reduced input for the vectors we tested. The reduction is therefore *conjecturally* redundant on this call site.

**What.** Delete the `poly_reduce(&w1)` call.

**So what.** **242,376** instructions (**5.20**×; −5,412). The bench verifies the secure-boot test vector across every ITERS we run with $g_{\mathrm{acc}} = 0$ (verifier returns success).

**Caveat.** This is the only optimization in the series whose correctness we have not algebraically established. Specifically, we do not prove that pqm4's `m4f` invNTT-tomont produces the correct output for every input in $[-5q, 5q]^n$ — only that for this one valid signature it does. A production deployment should either (a) tighten the bound by adding a worst-case analysis of the matrix–vector accumulator's Montgomery-domain output, (b) keep `poly_reduce`, or (c) replace the M4 invNTT with one that documents its admissible input range. We mark this as an open soundness item rather than a confirmed correctness result.

### 5.10 v10 — Raw Keccak Driving for $\tilde{c}'$

**Why.** The inc-API of `pqm4/mupq/common/fips202.c` wraps each Keccak operation with parameter marshalling and a position-tracking struct (`uint64_t ctx[26]`, where `ctx[25]` holds the position counter). Each `shake256_inc_absorb` call performs the position-counter bookkeeping; calling it $1 + k = 5$ times per verify adds plumbing.

**What.** We bypass the inc-API entirely for the $\tilde{c}'$ stream and call the underlying asm primitives — `KeccakF1600_StateXORBytes`, `KeccakF1600_StatePermute`, `KeccakF1600_StateExtractBytes` — directly. The position counter lives in a local `size_t pos`, kept in a register across the loop body.

**So what.** **241,756** instructions (**5.22**×; −621). The win is small because the inc-API plumbing wasn't large to begin with — but it retires the loose end from v8's failed direct-XOR experiment.

# 6 Results

## 6.1 Measurement Methodology

All numbers are *instructions retired* on the Cortex-M4 ISA as observed by a QEMU TCG plugin [5] running `qemu-system-arm 11.0.0 -M mps2-an386`. The benches are bare-metal ELFs compiled with `arm-none-eabi-gcc 13.2.1` (`-mcpu=cortex-m4 -mthumb -mfpu=fpv4-sp-d16 -mfloat-abi=hard -O3`). Instructions retired are a *lower bound* on real-silicon cycles — they count each retired instruction as one and ignore taken- branch refill, single-cycle vs. multi-cycle `LDR/STR` pairing, the 3–7-cycle range of `SMULL/UMULL/ SMLAL`, the 2–12-cycle `SDIV/UDIV`, and any flash wait states the device imposes. The metric is therefore best used for *comparing* implementations of the same algorithm rather than for absolute wall-clock prediction: a real-silicon-cycles to instructions-retired ratio of roughly 1.3×–2.5× is typical for cryptographic workloads on Cortex-M4, with the higher end applying to memory-bound code on slow flash and the lower end applying to register-rich loops in tightly-cached code. We cross-checked one point against published silicon numbers: pqm4 lists ∼1.65M cycles for ML-DSA-44 verify on STM32F407, vs. our 1,270,759 instructions retired for the same code, giving a ratio of ∼1.3.

Per-call cost is extracted as $(\text{insns}(\mathsf{ITERS} = 11) - \text{insns}(\mathsf{ITERS} = 1))/10$, which cancels program startup, the one-shot precompute, and the trailing semihosting writes. The methodology is bit-exactly reproducible because QEMU's TCG executes deterministically and our test inputs are hardcoded; we verified that the formula yields the same answer at $\mathsf{ITERS} = 101$.

## 6.2 The Table

Table 3: Per-verify cost on Cortex-M4, instructions retired. Every sb-mldsa44-v$N$ ($N \geq 2$) precomputes from pk once before the timed loop; the precompute cost is amortized away by the measurement formula.

| Variant | Insns retired | vs v1 | Δ vs prev. |
|---|---|---|---|
| v1 pqm4 m4f baseline | 1,261,555 | 1.00× | — |
| v2 + precompute $\hat{\mathbf{A}}$ | 410,292 | 3.07× | −67.5% |
| v3 + precompute NTT($2^d t_1$) | 368,019 | 3.43× | −10.3% |
| v4 + precompute $tr_{\mathsf{pk}}$ | 268,420 | 4.70× | −27.1% |
| v5 + pre-warmed $\mu$ Keccak | 267,944 | 4.71× | −0.2% |
| v6 + one-shot hint decode | 265,460 | 4.75× | −0.9% |
| v7 + caddq/UseHint/pack fusion | 256,559 | 4.92× | −3.4% |
| v8 + unpack/chknorm fusion | 247,788 | 5.09× | −3.4% |
| v9 – redundant `poly_reduce` | 242,376 | 5.20× | −2.2% |
| v10 + raw Keccak driver | **241,756** | **5.22**× | −0.3% |

### 6.3 Cross-Library Context

Placing v10 alongside the off-the-shelf verifiers we measured in earlier work (Table 4):

Table 4: Cross-library verify cost on Cortex-M4 (instructions retired under −O3 with the harness described in Section 6), with per-key storage and signature size alongside so the space-time trade is visible at a glance. The Dilithium2 entry follows the round-3 Dilithium specification rather than FIPS 204 ML-DSA-44, so it does not share byte-level test vectors with the ML-DSA entries; the algebra is identical and the instruction counts are directly comparable. P-256 entries include sig = raw $(r, s)$ and pk = uncompressed $(x, y)$ without a leading `0x04` tag.

| Implementation | pk (B) | sig (B) | Insns retired | vs v10 |
|---|---|---|---|---|
| sb-mldsa44-v10 (this work) | 20,688† | 2,420 | **241,756** | 1.00× |
| pqm4 m4f ML-DSA-44 (generic, M4 asm) | 1,312 | 2,420 | 1,270,759 | 5.26× |
| mldsa-native ML-DSA-44 (portable C90) | 1,312 | 2,420 | 2,120,554 | 8.77× |
| FasterKyberDilithiumM4 Dilithium2 "new" [7] | 1,312 | 2,420 | 1,335,713 | 5.52× |
| oreparaz/p256 ECDSA (pure C, incl. SHA-256) | 64 | 64 | 8,122,459 | 33.6× |
| p256-m ECDSA (M4 UMAAL asm) | 64 | 64 | 12,320,435 | 51.0× |

† Materialized precompute (16,384-byte $\hat{\mathbf{A}}$ + 4,096-byte NTT($2^d t_1$) + 208-byte pre-warmed SHAKE state); the original 1,312-byte FIPS-204 pk is no longer read at runtime.

The FasterKyberDilithiumM4 measurement is worth a remark. The repository's published wins are concentrated on the signing side (where the rejection loop dominates and the small-modulus NTTs are most useful); for verify the structure is essentially pqm4's loop with the matrix materialized on the stack rather than streamed. The 5% gap to `pqm4 m4f` we measure reflects that materialization plus minor compiler scheduling differences — not the underlying primitives, which are largely the same hand-tuned assembly.

## 7 Discussion

### 7.1 What This Is and Is Not

sb-mldsa44-v10 is a *deployment-specialized* verifier, not an algorithmic improvement to ML-DSA. The savings come from amortizing pk-derived computation across boots and from removing abstraction layers around the underlying primitives. It does *not* replace or weaken any FIPS-204 step: the spec-mandated norm bound on $\mathbf{z}$, the strong-unforgeability monotone-encoding check on $\mathbf{h}$, and the recompute-and-compare of $\tilde{c}'$ all remain in place.

It is also not a *constant-time* verifier: the timing of `sb_v8_unpack_sig_z_chknorm`'s early return, of `sb_v6_unpack_sig_h_all`'s monotone check, and of the hint-conditional branch inside `sb_v7_caddq_useHint_pack` all depend on the signature contents. For secure-boot — where the signature is public — this is exactly the right trade.

### 7.2 Tradeoffs: Space, Provisioning Time, and Brittleness

The cycle savings come at clear costs in other dimensions. We quantify them here so an integrator can decide if the trade fits their device.

**Expanded public-key footprint.** The native ML-DSA-44 public key is 1312 bytes (32-byte $\rho$ seed + $k = 4$ packed `POLYT1_PACKEDBYTES` = 320-byte $t_1$ components). What sb-mldsa44-v10 actually stores per provisioned key is the materialized precompute: the matrix $\hat{\mathbf{A}}$ in NTT-Montgomery form, the precomputed NTT($2^d \cdot t_1$), and a SHAKE-256 incremental-state template with $tr$ already absorbed (Table 5).

Table 5: Per-public-key storage required by each variant of the secure-boot verifier. ELF `.bss` sizes are reported by `arm-none-eabi-size` on the bench binaries; the precompute itself is in `.bss` in our QEMU harness but would live in flash on real silicon.

| Variant | Item | Bytes |
|---|---|---|
| v1 | FIPS-204 packed pk | 1,312 |
| v2 | $+ \hat{\mathbf{A}} \in \mathbb{Z}^{k\ell \cdot n}$ (NTT-Mont) | 16,384 |
| v3 | $+ \mathrm{NTT}(2^d t_1)_k \in \mathbb{Z}^{k \cdot n}$ (NTT-Mont) | 4,096 |
| v5 | + pre-warmed SHAKE-256 state (struct) | 208 |
| v10 | total (excluding the original 1312-byte pk, which the verifier no longer reads at runtime) | — |

That is a **15.8**× expansion over the original 1312-byte key. Concretely, sustaining $N$ provisioned trust-roots in flash (e.g. for key rotation, multi-vendor boot, or recovery) costs $N \cdot 20.2$ KB instead of $N \cdot 1.3$ KB. On a typical M4-class device with 256 KB–1 MB of internal flash this is usually affordable for $N \leq 10$; on a ∼64 KB-flash microcontroller it is not. Larger parameter sets scale poorly: applying the same recipe to ML-DSA-65 would cost ≈ 36.2 KB per key ($k\ell = 30$ matrix polynomials + $k = 6$ $t_1$ polynomials), and to ML-DSA-87 about **64.2** KB per key ($k\ell = 56$ matrix polynomials + $k = 8$ $t_1$ polynomials). Code size grows modestly: the `.text` segment is 728 bytes larger in v10 than in v1 (about +3.3%), absorbing the precompute function, the fused-tail loop, and the hand-driven Keccak helpers.

**One-time provisioning cost.** Computing the precompute table costs ≈ 1,004,000 instructions retired (measured as the ITERS = 0 baseline of v10 minus the 178-instruction startup of v1). On a 100 MHz M4 with a typical 1.3–2.5× cycles-per-instruction-retired ratio for Cortex-M4 cryptographic code, that is 13–25 ms of one-shot work at factory provisioning time — negligible compared to a device's manufacturing flash-write cost. Devices that allow secure-boot key rotation in the field would pay the same cost per rotation.

**Brittleness to upstream changes.** The precomputed $\hat{\mathbf{A}}$ and $\mathrm{NTT}(2^d t_1)$ tables are stored in `pqm4 m4f`'s internal NTT-Montgomery representation: each 32-bit limb is a particular signed residue modulo $q$ in the specific Montgomery-form, NTT-bit-reversed layout that `ntt.S` and `pointwise_mont.s` consume. A future pqm4 update that changes the bit-reversal layout, the Montgomery $R$, or the choice of $\zeta$ silently invalidates every stored precompute. Production deployments should pin the pqm4 commit hash alongside the precompute and re-derive on any toolchain bump — the same hygiene one applies to JIT cache files.

**No saving in signature size or transmission cost.** The signature itself is unchanged: $32 + \ell \cdot 576 + (\omega + k) = 2{,}420$ bytes for ML-DSA-44. If the bottleneck of the deployment is the signature transmission rather than verify cycles (e.g. a firmware update over a constrained radio), sb-mldsa44-v10 provides no help. For comparison: P-256 signatures are 64 bytes and Ed25519 signatures are 64 bytes; ML-DSA-44 is about 38× larger.

**Side-channel posture.** Several of the optimizations deliberately break the constant-time discipline that pqm4 maintains on the signing side: v6's monotone-encoding check exits on the first malformed byte; v7's tail branches on the hint bit and on the sign of $a_0$; v8's unpack-and-chknorm fast-rejects on the first out-of-range coefficient. None of these leak secret information — the verify-time inputs (pk, $\sigma$, $M$) are public — but a deployment that ports this verifier into a context where the signature is somehow private (e.g. a sealing/decryption use case) would need to revisit each fast-path.

**Per-verify dynamic memory.** The verify routine allocates its working polynomials on the stack: $\hat{\mathbf{z}}$ ($\ell \cdot n \cdot 4 = 4$ KB), $\mathbf{h}$-vector ($k \cdot n \cdot 4 = 4$ KB), the per-row temporaries ($2 \cdot n \cdot 4 = 2$ KB), the Keccak state buffers, and the smaller $\tilde{c}$, $c_2$, $\mu$ arrays. Stack high-water mark is approximately 12 KB, the same shape as `pqm4 m4f`'s baseline. The extra static `.bss` (20.2 KB) is on top of, not in place of, that stack budget.

### 7.3 The Long Tail vs. the Two Big Wins

The cumulative speedup ladder in Table 3 has a characteristic "two stairs and a slope" shape. The two `pk`-precompute transformations `v2` (matrix $\mathbf{A}$, 851,263 insns) and `v4` ($tr$ hash, 99,599 insns) together account for 950,862 of the total 1,019,799 instructions removed across all ten variants — 93.2% of the cumulative savings, or equivalently 75.4% of `v1`'s original cost. The remaining seven transformations (`v3` and `v5`–`v10`) contribute 68,937 instructions total, or 6.8% of cumulative savings (5.5% of `v1`). The lesson is that precomputing `pk`-derived data is the strategically important move; the rest are tactical fusions and de-abstractions.

### 7.4 What's Not Optimized (Limits of This Work)

We did not write any new Cortex-M4 assembly. The hot polynomial primitives — NTT, inverse NTT, pointwise-Mont, Keccak-$f[1600]$ — remain pqm4's `m4f` asm. A plausible next step would be a custom `poly_pointwise_sub_montgomery` that folds the $\hat{c} \cdot \mathrm{NTT}(2^d t_1)$ pointwise multiplication with the subsequent `poly_sub` from $\hat{\mathbf{w}}$ into a single fused pass, saving the round-trip through the intermediate `tmp_elem` polynomial. We do not estimate the achievable saving here.

### 7.5 What Remains on the Critical Path

The 241,756 instructions of `v10` are dominated by quantities that genuinely depend on per-verify inputs and that Table 2 therefore does not account for: the four forward NTTs on $\mathbf{z}$, one forward NTT on $c$, the $k\ell = 16$ pointwise-Montgomery accumulations of the matrix–vector product, the $k = 4$ inverse NTTs on $\hat{\mathbf{w}}'$, the SHAKE-256 absorbs that compute $\mu$ from the message, the SHAKE-256 stream that computes $\tilde{c}'$ from $\mu$ and $\mathbf{w}_1$, the `SampleInBall` of $c$, and the spec-mandated unpacking and norm/encoding checks. Estimating individual costs further would require selectively instrumenting each phase, which we have not done; we therefore present only what we can measure directly.

At a 100 MHz Cortex-M4 with typical wait-state penalties, 241,756 instructions retired corresponds to roughly 3–6 ms of wall-clock time. Comfortably inside the latency budget of any secure-boot deployment we are aware of.

## 8 Reproducibility

The source tree is at `https://github.com/oreparaz/mldsa-bench`. The QEMU plugin is from `cyclebench-m4` [5]; the underlying ML-DSA implementations are `mldsa-native` [4] (the C90 baseline) and `pqm4` [3] (the M4 asm baseline and the substrate for the secure-boot variants). After `./setup.sh`, `(cd cyclebench-m4 && make qemu)`, and `make -C cyclebench-m4/plugin`, the sweep is `(cd bench && ./run_sweep.sh)`.

## 9 Conclusion

ML-DSA-44 signature verification on a Cortex-M4 was already faster than P-256 ECDSA verification in every off-the-shelf configuration we benchmarked. By exploiting two structural properties of secure-boot deployments — a fixed public key and all-public inputs — we reduced the

cost by an additional 5.22×, to ∼242 k instructions retired per verify. The two pk-precomputation steps (v2 and v4) account for 75.4% of v1's original cost (or, equivalently, 93.2% of all instructions we removed across the ten variants); the remaining seven transformations contribute another 5.5% of original cost. The verifier reproduces the FIPS-204 accept/reject decision on the secure-boot input subset $\mathcal{I}_{\mathrm{sb}}$ defined in Section 4 — modulo the open soundness item flagged at v9 — and deliberately rejects every input outside that subset.

## References


[1] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, D. Stehlé. CRYSTALS-Dilithium: A Lattice-Based Digital Signature Scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2018(1), pp. 238–268.

[2] National Institute of Standards and Technology. FIPS 204: Module-Lattice-Based Digital Signature Standard. 2024. `https://csrc.nist.gov/pubs/fips/204/final`.

[3] M. J. Kannwischer, P. Schwabe, D. Stebila, T. Wiggers. Improving Software Quality in Cryptography Standardization Projects. *IEEE European Symposium on Security and Privacy Workshops*, 2022, pp. 19–30. `https://github.com/mupq/pqm4`.

[4] The Post-Quantum Cryptography Alliance. `mldsa-native`: a portable C90 implementation of ML-DSA. `https://github.com/pq-code-package/mldsa-native`.

[5] O. Reparaz. `cyclebench-m4`: A small harness for estimating Cortex-M4 cycle counts via QEMU's TCG plugin API. `https://github.com/oreparaz/cyclebench-m4`.

[6] M. Pégourié-Gonnard. `p256-m`: a minimalistic implementation of ECDH and ECDSA on NIST P-256. `https://github.com/mpg/p256-m`.

[7] `FasterKyberDilithiumM4`: a Cortex-M4 implementation of Kyber and Dilithium with further NTT and pointwise improvements over `pqm4`. `https://github.com/FasterKyberDilithiumM4/FasterKyberDilithiumM4`.